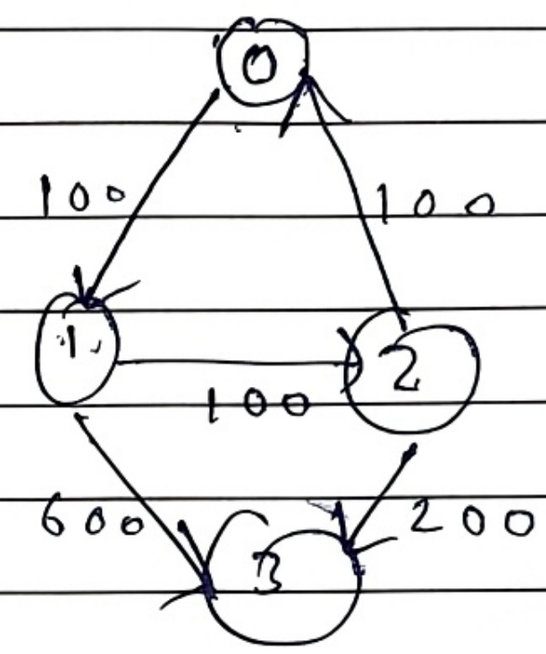
Name => Asmit Prabhakar
Uid => 23BCS10063

Subject => Competitive Programming.

Input = 4, flights = [[0,1,100] [0,2,100]], [[2,0,100], [1, 3, 600], [2, 3, 200]]

output = 700.

src = 0,
dst = 3,
k = 1.



Approach:-

i) First initialize distance array.
arr[i] = minimum cost to reach the node.
Set all = ∞, expect dist[src] = 0.

ii) create queue.
Store:(node, cost, stops)
push: {src, 0, 0}

iii) Start BFS Traversal.

iv) Explore neighbours.
For each neighbour
calculate new cost
if newcost < dist[neighbour]
→ update distance.

→ Push into queue with stops+1.

5) Return answer
 if dist[dst] == ∞
  return -1;
 else return dist[dst]

code :-

```
int findcheapestprice(int n, vector<vector
<int>> flight, int src, int dist int k)
{
 vector<vector<pair<int>> adj(n);

 for (auto & f: flight) {
  adj[f[0]].pushback({ f[1], f[2]});
 }
 vector<int> dist(n, INT_MAX);
  dist[src] = 0;
  queue<tuple<int, int, int>>q;
  q.Push({src, 0, 0});
 while (!q.empty()){
  auto [node, cost, stops] = q.front();
  q.pop();
  if ( stops > k)
   continue;
  for (auto & it : adj[node]){
```

```
int nextnode = it.first;
    int price = it.second;
    if(cost + price < dist[nextnode]){
        dist[nextnode] = cost + price;
        q.push({nextnode, dist[nextnode]});
            ~~dist~~ stops++;
    }
}
}
} return dist[dst] == INT_MAX ? -1 : dist[dst];
}
```

T.C = O(E*K)
S.C = O(V+E)